

Building on Maschina

September 20, 2026

Version 1.0

For developers: the interface, machine kinds, and using the custody layer on its own

Asher Wilson, Founder

asher@maschine.dev

Abstract

Maschina gives software a way to hold and spend money without being trusted with it. This document covers the public interface, its conventions, how a machine kind is written and why an untrusted one is harmless, what a developer inherits without writing it, publishing and capacity pricing, and how to use the custody layer from software that is not ours.

1. What you get

A wallet whose limits are enforced outside your code, a budget that settles at true cost, a permanent record, and execution that can be stopped mid flight.

If you are building anything that lets software touch money, this is the part you do not want to write yourself. It is also the part you will be blamed for when it fails, which is the more persuasive of the two reasons.

The providers converging on embedded wallets with policy engines have the right instinct [9]. Maschina differs in one respect that matters: the party being governed never holds the key at all.

2. The interface

<https://api.maschina.dev>

GET /openapi.json describes every route. Generate a client from it rather than writing one by hand.

Routes cover the whole life of a machine: create one with its own wallet, list them, read one, read its record, fund it, start, pause, resume and stop it.

Every route resolves the owner from the session and asks for “this owner’s machine with this identifier” in a single query. A machine that is not yours is not found, rather than refused with a message confirming that it exists.

3. Conventions

Amounts are strings of digits in base units, never floating point. Fifteen units of a six decimal asset is “15000000”. Times are ISO 8601 in UTC. An unset limit is absent rather than zero, because zero and unset mean opposite things. Errors carry a stable code, a human readable message and a request identifier worth quoting when you ask for help. Rate limits are per caller and reported in response headers.

4. Creating machines

Creation is the one call with irreversible external effects, so it is ordered and verified: the wallet is created, the policy is written, the policy is read back and compared, and only then is the machine recorded. Any mismatch means nothing exists.

You never supply the address that funds may return to. It comes from the session. This is not a convenience, it is the guarantee: a machine’s funds can only ever reach the person who created it, and no request body can change that.

5. Writing a machine kind

A kind is a deterministic decision function. Given settings, balances and history, it returns the action it believes should occur, or nothing.

It cannot execute, hold a key, or contact a venue. It proposes, and the proposal passes the mandate, the ledger and the wallet policy, each of which can refuse it independently.

This is why your kind does not have to be careful with money: it cannot reach any. It is also why an untrusted kind written by a stranger can be run on hardware owned by a different stranger without either of them trusting the other.

A kind declares three things.

1. **Its settings**, validated strictly. Missing or malformed is an error, never a default.
2. **Its decision**, deterministic in its inputs, so any party can replay a run and check it.
3. **What it observes**, for example a price level, so the platform knows when to wake it.

6. What you inherit without writing it

Machine state and window checks. Approved asset enforcement. Per action, daily and total caps. Budget reservation and settlement at true cost. Single send with signature reuse on retry. Quote validation against an independent price. Mandatory asset safety checks. The complete record, with your reasoning attached. Pause, stop, and global halt.

You cannot switch any of it off, including the machines you wrote yourself.

7. Testing

Deterministic decisions are trivial to test, which is the point of the design. Integration runs against real infrastructure rather than mocks, because the failures worth catching are the ones mocks cannot produce: concurrency, ordering, partial failure, and the gap between an estimate and reality.

A test that cannot fail has tested nothing. That is the standard the platform holds itself to, and the standard a published kind is held to.

8. Publishing

A kind or a configured machine can be published for others to copy. Copies run with the copier's funds, wallet and limits, never the author's. The author earns for as long as copies run.

Publishing requires a bond. If copies fail in ways the original's record never showed, copiers are compensated from it, and the amount is displayed beside the machine. A claim backed by the claimant's own money is the only rust signal in this industry worth displaying.

Capacity is finite and priced. Copy slots are limited and auctioned, so a strategy is not quietly destroyed by unlimited crowding into it.

9. Using the custody layer on its own

The custody and budget layer is available to software that is not ours. An agent, a service or an application can obtain a wallet with enforced limits, a budget and a record, and operate against them directly without touching the rest of the product.

This exists because the problem is general. Anything that lets software spend money has it, the payment standards now being adopted across the industry make it more urgent rather than less [8], and solving it once properly is more useful than solving it badly in fifty places.

10. Source

github.com/maschina-labs/maschina, public for review with all rights reserved. Source available rather than open source: read it, check it, hold us to it.

11. References

[8] The Linux Foundation. x402 Foundation announcement, April 2, 2026.

[9] Coinbase AgentKit, Crossmint, Privy and Turnkey. Embedded wallet providers offering policy engines that gate signatures against a principal's rules.